

GA-SUM-01 · PE-U1

Práctica Experimental — Unidad 1

Comunicación entre Procesos Distribuidos con gRPC y Sockets TCP

Trabajo Sumativo — GA · Semana 4 · 25–31 Mayo 2026 · Período Regular 2025-2026 SPA

Código	GA-SUM-01
Tipo de actividad	Aplicación de contenidos procedimentales
Unidad	1 — Generalidades de los Sistemas Distribuidos
Semana de entrega	4 — 25 al 31 de Mayo del 2026
Período académico	Regular 2025-2026 SPA
Modalidad	Grupal (equipos de 3-4 estudiantes)
Duración	3 horas de laboratorio + 5 horas autónomas
Entregable	Repositorio GitHub + Documento LaTeX compilado (PDF + .tex)
Integrante 1 / Rol	[Nombre — Arquitecto]
Integrante 2 / Rol	[Nombre — Desarrollador]
Integrante 3 / Rol	[Nombre — Resp. Calidad]
Integrante 4 / Rol	[Nombre — Documentalista]
URL Repositorio GitHub	https://github.com/equipo/ga-sum-01
Calificación (docente)	___ / 100

1. Tipo de Actividad

Tipo	Aplicación de contenidos procedimentales
Modalidad	Grupal (equipos de 3-4 estudiantes)
Duración	3 horas de laboratorio + 5 horas autónomas
Entregable	Repositorio GitHub + Documento LaTeX compilado (PDF + .tex)

2. Tema de la Práctica

Implementación y comparación de mecanismos de comunicación entre procesos distribuidos: sockets TCP/UDP y llamadas a procedimiento remoto (RPC) con gRPC. Análisis del comportamiento del sistema bajo condiciones de sincronización distribuida aplicando el algoritmo de relojes lógicos de Lamport.

3. Objetivos

3.1 Objetivo General

Implementar y comparar los mecanismos de comunicación entre procesos (sockets TCP y gRPC) en un sistema distribuido de tres nodos, analizando sus características de rendimiento y fiabilidad, aplicando el algoritmo de Lamport para ordenar eventos, y documentando los resultados con calidad académica en LaTeX.

3.2 Objetivos Específicos

- **OE1:** Configurar un entorno de desarrollo distribuido con al menos tres procesos que se comunican a través de una red local o contenedores Docker.
- **OE2:** Implementar un sistema de mensajería usando sockets TCP en Python y el mismo sistema usando gRPC, midiendo la latencia promedio de cada enfoque con 100 envíos.
- **OE3:** Aplicar el algoritmo de relojes lógicos de Lamport para ordenar los eventos del sistema distribuido implementado.
- **OE4:** Documentar el proceso, resultados y análisis en un documento LaTeX con referencias bibliográficas IEEE de calidad académica (mínimo 6 con DOI).

4. Fundamento Teórico

Instrucción obligatoria

Desarrollar en el documento LaTeX el fundamento teórico COMPLETO de la Unidad 1. Cada subtema: mínimo 300 palabras, al menos 2 referencias IEEE con DOI verificado, un diagrama propio en TikZ o figura importada, y código de ejemplo en Istlisting donde aplique.

4.1 Sockets TCP/UDP — Comunicación a Nivel de Transporte

Definición: Un socket es la interfaz entre el proceso de aplicación y la pila de protocolos de red del sistema operativo. Proporciona un punto de comunicación identificado por (dirección IP, puerto, protocolo).

Los sockets TCP (SOCK_STREAM) proveen: orientación a conexión (handshake de 3 vías), entrega garantizada y en orden, control de flujo y congestión. Los sockets UDP (SOCK_DGRAM) proveen: sin conexión, best-effort, sin garantía de orden ni entrega — pero menor overhead.

Comparación TCP vs UDP en SD

Característica	TCP	UDP
Conexión	Orientado (3-way handshake)	Sin conexión
Fiabilidad	Garantizada	Best-effort
Orden	Garantizado	No garantizado
Velocidad	Mayor overhead	Menor overhead
Uso en SD	APIs, transferencia archivos	Streaming, DNS, juegos

4.2 RPC y gRPC con Protocol Buffers

Remote Procedure Call (RPC) permite a un proceso ejecutar una función en otro proceso remoto como si fuera local. El stub del cliente serializa los parámetros y los envía por la red; el stub del servidor los deserializa, ejecuta la función y devuelve el resultado.

gRPC (Google RPC, 2015) implementa RPC usando Protocol Buffers como formato de serialización binaria y HTTP/2 como protocolo de transporte. Las ventajas respecto a REST/JSON: payload 3-10x más compacto, HTTP/2 multiplexado (múltiples streams sobre una TCP connection), y generación automática de código desde el archivo .proto.

4.3 Relojes Lógicos de Lamport (1978)

Lamport (1978) resuelve el problema de la imposibilidad de ordenar eventos en un SD sin reloj global mediante relojes lógicos basados en tres reglas:

- **Regla 1 (evento interno):** $LC = LC + 1$ antes de ejecutar el evento.
- **Regla 2 (envío):** $LC = LC + 1$; enviar el mensaje con $timestamp = LC$.
- **Regla 3 (recepción):** $LC = \max(LC_local, LC_recibido) + 1$.

Propiedad fundamental: Si el evento a causó el evento b ($a \rightarrow b$), entonces $LC(a) < LC(b)$. El recíproco no se cumple — timestamps iguales pueden ser de eventos concurrentes.

5. Procedimiento Completo

Paso 1 — Configuración del Entorno

Requisitos de software

Python 3.10+ · grpcio · grpcio-tools · protobuf · pandas · matplotlib · Git 2.x · GitHub account · LaTeX (TeX Live 2023 o Overleaf)

1. 1.1 Instalar las librerías Python:

```
# Terminal (ejecutar una sola vez)
pip install grpcio grpcio-tools protobuf pandas matplotlib

# Verificar instalación de grpc_tools
python -m grpc_tools.protoc --version
# Debe mostrar: 'libprotoc X.X.X' sin errores
```

2. 1.2 Crear el repositorio GitHub con la estructura requerida:

```
# Estructura del repositorio
ga-sum-01/
├── src/
│   ├── sockets/
│   │   ├── server.py
│   │   └── client.py
│   └── grpc/
│       ├── messaging.proto
│       ├── server_grpc.py
│       └── client_grpc.py
├── data/
│   ├── latency_sockets.csv
│   └── latency_grpc.csv
├── docs/
│   ├── guia1_practica.tex
│   ├── guia1_practica.pdf
│   └── figures/
│       └── boxplot.png
└── README.md
```

3. 1.3 Cada integrante debe hacer al menos 3 commits significativos. Ejemplos de commits aceptables:
- 'Implement LamportClock class with tick() and receive() methods'
 - 'Add TCP server with JSON payload and Lamport integration'
 - 'Generate latency benchmark data: 100 measurements for sockets and gRPC'
 - **NO aceptables:** 'Add files', 'Initial commit', 'Changes', 'Fix'

Paso 2 — Implementación con Sockets TCP

Archivo: src/sockets/lamport_clock.py

```
lamport_clock.py - Clase base (usar en AMBAS implementaciones)
class LamportClock:
    """Implementación de los relojes lógicos de Lamport (1978)."""

    def __init__(self, pid: int):
        self.pid = pid
        self.lc = 0

    def tick(self) -> int:
        """Reglas 1 y 2: evento interno o envío de mensaje.
        Incrementa el reloj ANTES del evento y retorna el nuevo valor."""
        self.lc += 1
```

```

        return self.lc

    def receive(self, received_ts: int) -> int:
        """Regla 3: recepción de mensaje.
        LC = max(LC_local, LC_recibido) + 1"""
        self.lc = max(self.lc, received_ts) + 1
        return self.lc

    def __repr__(self) -> str:
        return f"P{self.pid}[LC={self.lc}]"

```

Archivo: src/sockets/server.py

```

server.py – Servidor TCP con soporte Lamport
import socket, json, threading
from lamport_clock import LamportClock

HOST, PORT = '0.0.0.0', 5000
clock = LamportClock(pid=0) # El servidor es el proceso 0
lock = threading.Lock()

def handle_client(conn, addr):
    print(f'[SERVER] Conexión de {addr}')
    with conn:
        while True:
            data = conn.recv(4096)
            if not data:
                break
            msg = json.loads(data.decode())
            with lock:
                ts = clock.receive(msg['timestamp']) # Regla 3
                print(f'[SERVER] {clock} ← recv de P{msg["sender"]}')
            ts={msg["timestamp"]}
            response = json.dumps({'status': 'OK', 'server_ts': ts}).encode()
            conn.sendall(response)

def start():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((HOST, PORT))
        s.listen(10)
        print(f'[SERVER] Escuchando en {HOST}:{PORT}')
        while True:
            conn, addr = s.accept()
            t = threading.Thread(target=handle_client, args=(conn, addr))
            t.daemon = True
            t.start()

if __name__ == '__main__':
    start()

```

Archivo: src/sockets/client.py

```

client.py – Cliente TCP con Lamport + medición de latencia
import socket, json, time, csv
from lamport_clock import LamportClock

HOST, PORT = '127.0.0.1', 5000

def send_message(pid: int, content: str, clock: LamportClock) -> dict:
    """Envía un mensaje con timestamp Lamport y retorna la respuesta del
    servidor."""
    ts = clock.tick() # Regla 2: incrementar ANTES de enviar
    payload = json.dumps({

```

```

        'sender': pid,
        'timestamp': ts,
        'content': content
    }).encode()
    with socket.create_connection((HOST, PORT)) as s:
        s.sendall(payload)
        response = json.loads(s.recv(4096).decode())
    clock.receive(response['server_ts']) # Regla 3
    return response

def benchmark(pid: int, n: int = 100) -> list[float]:
    """Mide la latencia de n envíos. Retorna lista de latencias en ms."""
    clock = LamportClock(pid=pid)
    latencies = []
    for i in range(n):
        start = time.perf_counter() # ⚠ NO usar time.time()
        send_message(pid, f'msg_{i}', clock)
        end = time.perf_counter()
        latencies.append((end - start) * 1000) # convertir a ms
    return latencies

def save_csv(latencies: list[float], filename: str):
    with open(filename, 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['latency_ms'])
        for lat in latencies:
            writer.writerow([round(lat, 4)])

if __name__ == '__main__':
    import sys
    pid = int(sys.argv[1]) if len(sys.argv) > 1 else 1

    # Intercambios demostrativos (20)
    clock = LamportClock(pid=pid)
    print(f'=== Intercambios demostrativos (P{pid}) ===')
    for i in range(20):
        resp = send_message(pid, f'mensaje_{i}', clock)
        print(f'    {clock} → server_ts={resp["server_ts"]}')

    # Benchmark (100 envíos)
    print('\n=== Benchmark de latencia (100 envíos) ===')
    lats = benchmark(pid, n=100)
    save_csv(lats, f'../data/latency_sockets.csv')
    print(f'Media: {sum(lats)/len(lats):.3f} ms | Min: {min(lats):.3f} | Max: {max(lats):.3f}')
    print('CSV guardado en data/latency_sockets.csv')

```

Paso 3 — Implementación con gRPC

Archivo: src/grpc/messaging.proto

```

messaging.proto — Contrato de la API (Protocol Buffers v3)
syntax = "proto3";
package messaging;

// Servicio principal de mensajería distribuida
service MessagingService {
    // RPC unario: el cliente envía un Request, el servidor responde con Response
    rpc SendMessage (Request) returns (Response);
}

// Mensaje de solicitud del cliente
message Request {
    string sender = 1; // Identificador del proceso cliente (PID)
    int64 timestamp = 2; // Reloj de Lamport del cliente en el momento del envío
}

```

```

    string content    = 3;    // Contenido del mensaje
}

// Mensaje de respuesta del servidor
message Response {
    string status      = 1;    // 'OK' o código de error
    int64  server_ts   = 2;    // Reloj de Lamport del servidor después de receive()
}

```

4. 3.1 Generar el código Python desde el .proto (ejecutar en src/grpc/):

```

# Generar stubs Python desde messaging.proto
cd src/grpc

python -m grpc_tools.protoc \
    -I. \
    --python_out=. \
    --grpc_python_out=. \
    messaging.proto

# Verificar que se crearon los archivos:
ls -la
# Debe mostrar: messaging_pb2.py y messaging_pb2_grpc.py

```

5. 3.2 Implementar el servidor gRPC:

Archivo: src/grpc/server_grpc.py

```

server_grpc.py - Servidor gRPC con Lamport
import grpc
from concurrent import futures
import messaging_pb2
import messaging_pb2_grpc
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '../sockets'))
from lamport_clock import LamportClock

clock = LamportClock(pid=0) # Servidor = proceso 0

class MessagingServiceServicer(messaging_pb2_grpc.MessagingServiceServicer):
    def SendMessage(self, request, context):
        """Implementación del RPC SendMessage."""
        ts = clock.receive(request.timestamp) # Regla 3
        print(f'[gRPC SERVER] {clock} ← recv de {request.sender}')
        ts={request.timestamp}')
        return messaging_pb2.Response(
            status='OK',
            server_ts=ts
        )

def serve(port: int = 50051):
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    messaging_pb2_grpc.add_MessagingServiceServicer_to_server(
        MessagingServiceServicer(), server
    )
    server.add_insecure_port(f'[::]:{port}')
    server.start()
    print(f'[gRPC SERVER] Iniciado en puerto {port}')
    server.wait_for_termination()

if __name__ == '__main__':
    serve()

```

Archivo: src/grpc/client_grpc.py

```

client_grpc.py - Cliente gRPC con Lamport + benchmark
import grpc, time, csv, sys, os
import messaging_pb2
import messaging_pb2_grpc
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '../sockets'))
from lamport_clock import LamportClock

SERVER_ADDR = 'localhost:50051'

def send_message(pid: str, content: str, clock: LamportClock,
                 stub: messaging_pb2_grpc.MessagingServiceStub) ->
messaging_pb2.Response:
    ts = clock.tick() # Regla 2
    request = messaging_pb2.Request(
        sender=pid,
        timestamp=ts,
        content=content
    )
    response = stub.SendMessage(request)
    clock.receive(response.server_ts) # Regla 3
    return response

def benchmark(pid: str, n: int = 100) -> list[float]:
    channel = grpc.insecure_channel(SERVER_ADDR)
    stub = messaging_pb2_grpc.MessagingServiceStub(channel)
    clock = LamportClock(pid=int(pid))
    latencies = []
    try:
        for i in range(n):
            start = time.perf_counter()
            send_message(pid, f'msg_{i}', clock, stub)
            end = time.perf_counter()
            latencies.append((end - start) * 1000)
    finally:
        channel.close()
    return latencies

if __name__ == '__main__':
    pid = sys.argv[1] if len(sys.argv) > 1 else 'P1'
    channel = grpc.insecure_channel(SERVER_ADDR)
    stub = messaging_pb2_grpc.MessagingServiceStub(channel)
    clock = LamportClock(pid=int(pid[-1]))

    # Intercambios demostrativos
    print(f'=== Intercambios demostrativos ({pid}) ===')
    for i in range(20):
        resp = send_message(pid, f'mensaje_{i}', clock, stub)
        print(f' {clock} → server_ts={resp.server_ts}')
    channel.close()

    # Benchmark
    print('\n=== Benchmark gRPC (100 envíos) ===')
    lats = benchmark(pid, n=100)
    with open('../data/latency_grpc.csv', 'w', newline='') as f:
        csv.writer(f).writerows(['latency_ms'] + [[round(l,4)] for l in lats])
    print(f'Media: {sum(lats)/len(lats):.3f} ms | Min: {min(lats):.3f} | Max: {max(lats):.3f}')
    print('CSV guardado en data/latency_grpc.csv')

```

Paso 4 — Análisis Comparativo

Archivo: analysis.py (ejecutar desde la raíz del proyecto)

```

analysis.py - Estadísticas + boxplot
import pandas as pd

```



```

import matplotlib.pyplot as plt
import matplotlib
matplotlib.use('Agg') # Evitar errores de display en entornos sin GUI

def analyze():
    # Cargar datos
    sockets = pd.read_csv('data/latency_sockets.csv')['latency_ms']
    grpc = pd.read_csv('data/latency_grpc.csv')['latency_ms']

    df = pd.DataFrame({'Sockets TCP': sockets, 'gRPC': grpc})

    # Estadísticas completas
    print('='*55)
    print('ANÁLISIS ESTADÍSTICO COMPARATIVO - GA-SUM-01')
    print('='*55)
    stats = df.describe(percentiles=[.25, .50, .75, .95])
    print(stats.round(4).to_string())
    print()

    # Resumen ejecutivo
    for col in df.columns:
        print(f'{col}:')
        print(f'  Media:    {df[col].mean():.4f} ms')
        print(f'  Mediana:  {df[col].median():.4f} ms')
        print(f'  Std:      {df[col].std():.4f} ms')
        print(f'  P95:      {df[col].quantile(0.95):.4f} ms')
        print()

    # Boxplot comparativo (300 DPI para LaTeX)
    fig, ax = plt.subplots(figsize=(8, 5))
    df.boxplot(ax=ax, grid=True)
    ax.set_title('Comparación de Latencia: Sockets TCP vs gRPC', fontsize=14,
fontweight='bold')
    ax.set_ylabel('Latencia (ms)', fontsize=12)
    ax.set_xlabel('Tecnología de Comunicación', fontsize=12)
    plt.tight_layout()
    plt.savefig('docs/figures/boxplot.png', dpi=300, bbox_inches='tight')
    print('Boxplot guardado en docs/figures/boxplot.png (300 DPI)')
    plt.close()

if __name__ == '__main__':
    analyze()

```

Resultado esperado del análisis

El boxplot mostrará que gRPC tiene latencia media ligeramente MAYOR que sockets TCP en localhost. El overhead de HTTP/2 y Protocol Buffers es significativo en distancias de red cero. En red real (WAN), gRPC es mas eficiente por el payload mas compacto. Los datos del análisis deben respaldar esta hipótesis con P95.

Paso 5 — Documento LaTeX

Estructura mínima requerida del documento LaTeX

Sección 1: Portada — título, autores, códigos, período, fecha.

Sección 2: Resumen bilingüe — Abstract en inglés y Resumen en español, 200 palabras cada uno.

Sección 3: Fundamento Teórico U1 — 4 subsecciones (SD, Sockets/gRPC, Lamport, CAP). Mín. 300 palabras c/u con ≥ 2 referencias IEEE con DOI.

Sección 4: Arquitectura del Sistema — Diagrama TikZ de los 3 nodos con sus puertos y protocolos.

Sección 5: Implementación — Código en entornos `lstlisting` (NO pegar texto plano).

Sección 6: Resultados — Tabla `booktabs` con estadísticas + `boxplot` como `\includegraphics` con `\caption` y `\label`.

Sección 7: Análisis — 4 preguntas obligatorias respondidas con datos propios.

Sección 8: Conclusiones — Una conclusión individual por integrante (≥ 150 palabras c/u).

Bibliografía: ≥ 6 referencias IEEE con DOI verificado. Usar `\bibliographystyle{IEEEtran}`.

⚠ Advertencias críticas del documento LaTeX

REGLA 1: Un documento que NO compila con `pdflatex` tiene calificación 0 en el criterio LaTeX. Verificar en Overleaf o localmente antes de subir.

REGLA 2: No insertar imágenes de código. TODO el código debe estar en entornos `lstlisting` correctamente configurados.

REGLA 3: Las figuras deben tener `\caption` y `\label` referenciadas en el texto con `\ref{fig:boxplot}`. Una figura sin referencia en texto es decorativa, no cuenta.

6. Materiales, Equipos y Software

6.1 Equipos Mínimos

- Computadora con procesador Intel Core i5 o equivalente AMD, mínimo 8 GB RAM, 20 GB espacio libre.
- Acceso a red local (LAN) o Internet para repositorio GitHub.

6.2 Software Requerido

Software	Versión / Notas
Python	3.10 o superior
grpcio + grpcio-tools	Última versión (pip install)
protobuf	Última versión compatible con grpcio-tools
pandas + matplotlib	Última versión estable
Docker Desktop	Opcional para contenedores (alternativa a localhost)
Visual Studio Code o PyCharm	IDE recomendado
Git 2.x + cuenta GitHub	Repositorio público
TeX Live 2023 o MiKTeX	Alternativa: Overleaf (online, recomendado)
Google Colab	Alternativa si no hay Python local

6.3 Recursos en Línea

- **gRPC Python:** <https://grpc.io/docs/languages/python/>
- **Protocol Buffers:** <https://protobuf.dev/programming-guides/proto3/>
- **Lamport 1978:** <https://doi.org/10.1145/359545.359563> (acceso gratuito via ACM DL)
- **Overleaf LaTeX:** <https://www.overleaf.com> (plantilla IEEEtran disponible)

7. Preguntas Obligatorias de Análisis

Las siguientes preguntas deben responderse en la Sección 7 del documento LaTeX, con apoyo en los datos experimentales obtenidos. Las respuestas sin datos propios no son aceptables.

Pregunta 1: Tiempo de convergencia vs. número de nodos

¿Cómo afecta el número de nodos al tiempo de convergencia de los relojes Lamport? Justifica con datos medidos. Considera el caso de 2, 3 y (teóricamente) 5 nodos. ¿La latencia de sincronización crece linealmente con el número de nodos?

Pregunta 2: Selección de protocolo para el PFC

¿En qué escenario del PFC usarías sockets TCP y en cuál gRPC? Argumenta desde la perspectiva del teorema CAP. Considera las propiedades CP vs AP de cada protocolo cuando el sistema está bajo partición de red.

Pregunta 3: Fallo de un nodo y relojes Lamport

Si un nodo falla durante la comunicación, ¿qué ocurre con los timestamps de Lamport en los nodos restantes? ¿El sistema puede continuar operando? ¿Los timestamps quedan en un estado inconsistente? Justifica con la Regla 3.

Pregunta 4: Transparencias ANSA logradas

¿Qué transparencias ANSA logra el sistema implementado? ¿Cuáles no se logran y por qué? Para cada transparencia no lograda, propone la modificación mínima al código que la implementaría.

8. Bibliografía Mínima (Norma IEEE)

6. [1] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," Commun. ACM, vol. 21, no. 7, pp. 558-565, Jul. 1978. doi: 10.1145/359545.359563
7. [2] M. van Steen and A. S. Tanenbaum, Distributed Systems, 4th ed., ver. 4.03. Maastricht: M. van Steen, Jan. 2025. ISBN: 978-90-815406-3-6. Available: <https://www.distributed-systems.net>
8. [3] K. Indrasiri and D. Kuruppu, gRPC: Up and Running. Sebastopol: O'Reilly, 2020. ISBN: 978-1-4920-5728-4.
9. [4] E. Brewer, "CAP twelve years later: How the 'rules' have changed," IEEE Comput., vol. 45, no. 2, pp. 23-29, Feb. 2012. doi: 10.1109/MC.2012.37
10. [5] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, Distributed Systems: Concepts and Design, 5th ed. Boston: Addison-Wesley, 2012. ISBN: 978-0-13-214301-1.
11. [6] B. Burns, Designing Distributed Systems, 2nd ed. Sebastopol: O'Reilly, 2025. ISBN: 978-1-09-815635-0.

Anexo A — Ejecución Paso a Paso

Secuencia de comandos completa para ejecutar la práctica

TERMINAL 1 — Servidor TCP:

```
cd src/sockets
python server.py
# Esperar: [SERVER] Escuchando en 0.0.0.0:5000
```

TERMINAL 2 — Cliente TCP (benchmark):

```
cd src/sockets
python client.py 1
# Ejecuta 20 intercambios demostrativos + 100 mediciones
# Guarda data/latency_sockets.csv
```

TERMINAL 3 — Servidor gRPC:

```
cd src/grpc
# (Primero generar el código si no se hizo)
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=.
messaging.proto
python server_grpc.py
# Esperar: [gRPC SERVER] Iniciado en puerto 50051
```

TERMINAL 4 — Cliente gRPC (benchmark):

```
cd src/grpc
python client_grpc.py P1
# Ejecuta 20 intercambios + 100 mediciones
# Guarda data/latency_grpc.csv
```

TERMINAL 5 — Análisis:

```
# Desde la raíz del proyecto
python analysis.py
# Genera stats + docs/figures/boxplot.png
```